

Mô tả Tổng Quan Hệ Thống gửi và nhận dữ liệu mã hóa bởi AES-CBC qua Socket

1 Tổng quan hệ thống

1.1 Hai thành phần chính

- **Sender (Client)**: đọc dữ liệu (text hoặc file), tạo AES key và IV, mã hóa rồi gửi qua socket.
- **Receiver (Server)**: lắng nghe socket, nhận AES key, IV và ciphertext, giải mã rồi lưu hoặc hiển thị kết quả.

1.2 Mạng LAN

Hai máy nằm cùng một mạng nội bộ, sử dụng địa chỉ IP và port cố định. Giao thức TCP đảm bảo truyền tin cậy.

2 Quy trình trao đổi

2.1 Khởi tạo kết nối dữ liệu

- Sender mở socket TCP: `connect(data_server_ip, data_port)`.
- Server: `bind(data_port) + listen()` và chấp nhận kết nối via `accept()`.

2.2 Giả lập kênh trao đổi AES key và IV

- Mở socket TCP riêng (cùng server IP nhưng `key_port`) để mô phỏng kênh bảo mật.
- Sender gửi AES key (16 hoặc 32 bytes) và IV (16 bytes) dưới dạng plaintext.
- Chú ý: Đây là kênh giả lập, chỉ để truyền thông tin mã hóa AES.
- Server nhận và lưu AES key, IV.

2.3 Gửi dữ liệu mã hóa

- Sender mã hóa dữ liệu thành ciphertext với key và IV đã trao.
- Gửi header độ dài (4–8 bytes) rồi ciphertext qua kênh dữ liệu.

2.4 Giải mã và xử lý

- Server đọc header độ dài và ciphertext.
- Dùng AES giải mã, sau đó lưu file hoặc in bản tin gốc.

2.5 Đóng kết nối

Cả hai phía gọi `close()` trên cả hai kênh.

3 Socket: Khái niệm và vai trò

- Socket là điểm kết nối mạng, trừu tượng hóa giao tiếp TCP/UDP.
- **TCP Socket**: đảm bảo truyền dữ liệu tin cậy, tuần tự và không mất gói.

4 Tham số và yêu cầu để gửi file

Table 1: Các thông số khởi tạo và yêu cầu

Tham số/Yêu cầu	Mô tả
<code>data_server_ip</code>	IP hoặc hostname máy nhận dữ liệu
<code>data_port</code>	Cổng TCP cho kênh dữ liệu (ví dụ 5000)
<code>key_port</code>	Cổng TCP cho kênh khóa (ví dụ 5001)
AES key size	16 bytes (AES-128) hoặc 32 bytes (AES-256)
IV	16 bytes ngẫu nhiên (CBC/CTR)
Padding	PKCS#7 khi dùng CBC; không cần khi CTR/GCM
Length header	4–8 byte integer (network byte order)
Chunk size	Với file lớn, nên gửi theo từng chunk (4KB–64KB)
Timeouts	Timeout cho <code>connect()</code> và <code>recv()</code>
Error handling	Bắt exception, kiểm tra lỗi mạng và timeout

5 Các cải tiến có thể

- Sử dụng AES-GCM để đồng thời mã hóa và xác thực.
- Thực hiện gửi file thay vì nhập từ bàn phím.
- Triển khai song song hai kênh TCP: kênh dữ liệu và kênh khóa.
- Nén dữ liệu trước khi mã hóa để tiết kiệm băng thông.
- Xây dựng giao thức ứng dụng nâng cao: xác thực người dùng, đa luồng, kiểm tra tính toàn vẹn.